

OE-EC604C — Object Oriented Programming

Semester VI • 3L:0T:0P • 3 Credits

Module 7 Exam Notes: Polymorphism

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Fundamentals of Polymorphism	2
1.1	Pointers to Objects and Polymorphic Assignments	2
1.2	The <code>this</code> Pointer	2
2	Compile-Time vs. Run-Time Polymorphism	2
2.1	Virtual Functions and Late Binding	2
3	Pure Virtual Functions and Abstract Classes	4
3.1	Pure Virtual Functions	4
3.2	Abstract Classes	4
4	Virtual Destructors & Memory Leaks	5
4.1	Destructor Call Failures	5

1. Fundamentals of Polymorphism

Polymorphism ("many forms") allows a single interface to represent different underlying forms (behaviors). In C++, it is broadly classified into compile-time and run-time polymorphism.

1.1 Pointers to Objects and Polymorphic Assignments

Pointers can store the addresses of objects. In inheritance hierarchies, C++ supports upcasting:

Upcasting Rule: A base class pointer can point to a derived class object without explicit typecasting. However, using this pointer, we can only access base class members, unless the functions are virtual.

```
Base* ptr = new Derived(); // Polymorphic Assignment (1)
```

1.2 The `this` Pointer

The `this` pointer is an implicit parameter passed to all non-static member functions. **Characteristics:**

- It stores the address of the calling object.
- Used to resolve naming conflicts between member variables and parameters: `this->data = data;`
- Used to return the calling object itself to enable **method chaining**: `return *this;`

2. Compile-Time vs. Run-Time Polymorphism

The primary distinction lies in when the compiler resolves function invocations.

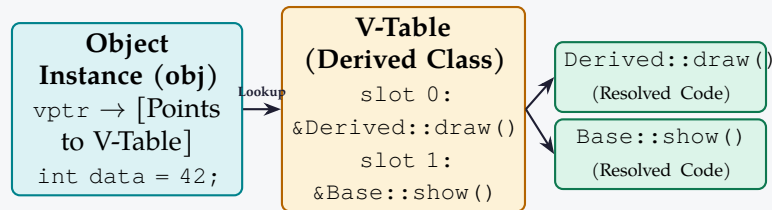
2.1 Virtual Functions and Late Binding

By prefixing a base class function declaration with the `virtual` keyword, we tell the compiler to resolve function calls at **run time** (late binding) based on the type of the object pointed to, rather than the type of the pointer.

Mechanism of Late Binding (V-Table and V-Ptr):

1. **V-Ptr (Virtual Pointer):** Every object of a class containing virtual functions is automatically allocated a hidden pointer field (`vptr`) pointing to its class's V-Table.
2. **V-Table (Virtual Method Table):** A static table compiled for each class containing function addresses for all its virtual functions.

Virtual Method Table (V-Table) Mapping Layout



Code Example: Runtime polymorphism using base pointer arrays

```

#include <iostream>
class Shape {
public:
    virtual void draw() { // Virtual function
        std::cout << "Drawing generic shape" << std::endl;
    }
    virtual ~Shape() {} // Virtual destructor
};

class Circle : public Shape {
public:
    void draw() override { // Overridden function
        std::cout << "Drawing Circle" << std::endl;
    }
};

class Box : public Shape {
public:
    void draw() override {
        std::cout << "Drawing Box" << std::endl;
    }
};

int main() {
    Shape* shapes[2];
    shapes[0] = new Circle();
    shapes[1] = new Box();

    for(int i = 0; i < 2; i++) {
        shapes[i]->draw(); // Late binding: calls appropriate draw()
        delete shapes[i];
    }
    return 0;
}
  
```

3. Pure Virtual Functions and Abstract Classes

Sometimes, a base class is so generic that it cannot provide any logical default implementation for its virtual functions.

3.1 Pure Virtual Functions

Pure Virtual Function: A virtual function declared in a base class with no implementation, represented by assigning = 0 at its declaration.

Declaration: `virtual Return_Type FuncName(Args) = 0;` (2)

3.2 Abstract Classes

A class containing at least one pure virtual function is an **Abstract Class**. **Key Characteristics:**

- **Instantiation:** Abstract classes cannot be used to instantiate objects. E.g., `Employee emp;` is a compile error.
- **Subclassing:** Derived classes must override **all** inherited pure virtual functions to become concrete classes. Failure to do so makes the derived class abstract as well.

Code Example: Abstract Employee base class

```
#include <iostream>
#include <string>
class Employee {
protected:
    std::string name;
public:
    Employee(std::string n) : name(n) {}
    virtual double calculateSalary() = 0; // Pure virtual function
    virtual void display() {
        std::cout << "Employee: " << name << std::endl;
    }
    virtual ~Employee() {}
};

class FullTimeEmployee : public Employee {
private:
    double monthlySalary;
public:
    FullTimeEmployee(std::string n, double s) : Employee(n), monthlySalary(s) {}
    double calculateSalary() override { return monthlySalary; }
};
```

4. Virtual Destructors & Memory Leaks

When destroying dynamically allocated derived class objects through base class pointers, memory leaks will occur unless the base class destructor is virtual.

4.1 Destructor Call Failures

If the base class destructor is **non-virtual**, the compiler binds the destructor call to the base pointer type at compile time. As a result, only the base class destructor executes. The derived class destructor never runs, leaving any heap allocations or handles inside the derived class unreleased.

Code Example: Memory leak prevention via Virtual Destructors

```
#include <iostream>
class Base {
public:
    Base() { std::cout << "Base Constructor" << std::endl; }
    // Declaring destructor virtual guarantees derived deallocation
    virtual ~Base() { std::cout << "Base Destructor" << std::endl; }
};

class Derived : public Base {
private:
    int* data;
public:
    Derived() {
        std::cout << "Derived Constructor" << std::endl;
        data = new int[100]; // Heap allocation
    }
    ~Derived() {
        std::cout << "Derived Destructor (Releasing Heap)" << std::endl;
        delete[] data; // Releases resource safely
    }
};

int main() {
    Base* ptr = new Derived();
    delete ptr; // Triggers Derived destructor then Base destructor
    return 0;
}
```

Quick Revision — Module VII

Polymorphism | OE-EC604C

Key Polymorphic Syntax

Virtual Declaration	<code>virtual void print(); // Inside base class declaration</code>
Override Qualifier	<code>void print() override; // Inside derived class (C++11)</code>
Pure Virtual Function	<code>virtual void process() = 0; // Defines abstract interface</code>
Virtual Destructor	<code>virtual ~Base() { /* virtual base destructor */ }</code>

One-Line Definitions

- **Polymorphism:** The ability of different classes to respond uniquely to identical message calls.
- **Virtual Function:** A class member function overridden in derived classes to ensure late binding execution.
- **Late Binding:** Resolving function calls at run time based on the active object's type.
- **V-Table:** A compiler-generated static lookup table storing virtual function addresses for a class.
- **V-Ptr:** An object instance pointer referencing the V-Table allocated during object instantiation.
- **Pure Virtual Function:** An abstract base method initialized to = 0 that derived classes must override.
- **Abstract Class:** A class containing pure virtual methods that cannot be instantiated.
- **Virtual Destructor:** A base class destructor configured to trigger proper intermediate subclass destructor sequences.
- **This Pointer:** An implicit member pointer storing the memory address of the calling object.

Frequently Asked / Viva Questions

Q1. What is the difference between early binding and late binding?

Early binding (compile time) resolves function addresses using pointer types. Late binding (run time) resolves addresses using the pointed-to object type via the V-Table lookup.

Q2. Can we declare a virtual function inside a structure (`struct`)?

Frequently Asked / Viva Questions (contd.)

Yes. Structures in C++ are identical to classes but use public access by default. They support the `virtual` keyword and inheritance.

Q3. What is the size impact of adding virtual functions to a class?

It adds the size of a single pointer (`vptr`) to every object instance (typically 4 bytes on 32-bit and 8 bytes on 64-bit systems), regardless of how many virtual functions are defined.

Q4. Can a class have virtual static member functions?

No. Static member functions are class-wide and do not bind to object instances. Since they lack a `this` pointer and `vptr` links, they cannot be virtual.

Q5. Can we override private virtual functions in C++?

Yes. The derived class can override private base virtual functions. Access levels only restrict calling permissions, not inheritance resolution.

Q6. What is the difference between overloading and overriding?

Overloading occurs within the same class scope (same name, different arguments). Overriding occurs between base and derived classes (same name, identical arguments, uses `virtual`).

Q7. Can an abstract class have constructors?

Yes. An abstract class cannot be instantiated directly, but its constructors run during derived class constructor sequences to initialize inherited data members.

Q8. Why does a constructor declare base classes virtual during multiple derivations?

To bypass duplicate grandparent member allocations in hybrid structures (solving the Diamond Problem).

Q9. What happens if we forget to declare base destructors virtual?

Deleting derived class instances through base class pointers only calls the base destructor, leaking derived heap resources.

Q10. What is a pure abstract class or interface?

A class that contains *only* pure virtual functions and no data members. It acts purely as a behavior contract.

Q11. Can we call a virtual function from a constructor or destructor?

Yes, but early binding is used. It calls the local class's implementation, not the derived class's version, because the derived object is not yet fully constructed or is already destroyed.

Q12. What is the purpose of the `override` keyword in C++11?

It instructs the compiler to verify that the function matches a virtual base class signature, preventing signature mismatch errors.

Comparison: Compile-Time vs. Run-Time Polymorphism

Feature	Compile-Time Polymorphism	Run-Time Polymorphism
Binding Time	resolved at compile time (Static Binding / Early Binding).	resolved at run time (Dynamic Binding / Late Binding).
Mechanism	Function Overloading and Operator Overloading.	Virtual Functions.
Speed	Fast execution (no run-time address lookup overhead).	Slightly slower (requires dereferencing the V-Ptr and V-Table lookup).
Flexibility	Fixed logic flow determined during compilation.	Highly flexible; handles new derived classes at runtime.
Requirements	Unique signatures (number/type of parameters).	Identical signatures; requires base pointer/reference interface.